Coaching How To Search

ICAART 2026

Vassilis Markos, and Loizos Michael
vassileiosmarkos@gmail.com, loizos@ouc.ac.cy

March 2026

Contents

Motivation

- Consider the following examples of AI agents:

Motivation

- Consider the following examples of AI agents:
  - Agent *A* provides machine translation services.

Motivation

- Consider the following examples of AI agents:
  - Agent A provides machine translation services.
  - Agent B provides "study at home" support for students.

Motivation

- Consider the following examples of AI agents:
  - Agent A provides machine translation services.
  - Agent B provides "study at home" support for students.
  - Agent C serves as a cognitive substitute of a therapist for their clients.

Motivation

- Consider the following examples of AI agents:
  - Agent A provides machine translation services.
  - Agent B provides "study at home" support for students.
  - Agent C serves as a cognitive substitute of a therapist for their clients.
- While agent A is typically focused on *what* should be achieved, agents B and, mostly, C, should also put emphasis on *how* their goal is reached.

Motivation

- Consider the following examples of AI agents:
  - Agent A provides machine translation services.
  - Agent B provides "study at home" support for students.
  - Agent C serves as a cognitive substitute of a therapist for their clients.
- While agent A is typically focused on *what* should be achieved, agents B and, mostly, C, should also put emphasis on *how* their goal is reached.
- Hence, faithful knowledge transfer arises as a natural prerequisite for tasks where an agent's behavior does matter.

Optimization as a Search Problem

- Such cases can be naturally encoded as search problems in a state space, where the agent searches for an optimal path, p, from a starting state, s, to a goal state, g.

Optimization as a Search Problem

- Such cases can be naturally encoded as search problems in a state space, where the agent searches for an optimal path, p, from a starting state, s, to a goal state, g.
- Thus, the *what* aspect corresponds to the goal state, g, the agent has to reach.

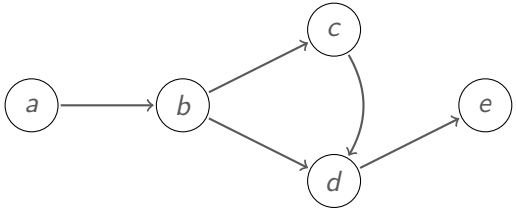
Optimization as a Search Problem

- Such cases can be naturally encoded as search problems in a state space, where the agent searches for an optimal path, p, from a starting state, s, to a goal state, g.
- Thus, the *what* aspect corresponds to the goal state, g, the agent has to reach.
- Similarly, the *why* aspect corresponds to the shape of the optimal path, p.

Optimization as a Search Problem

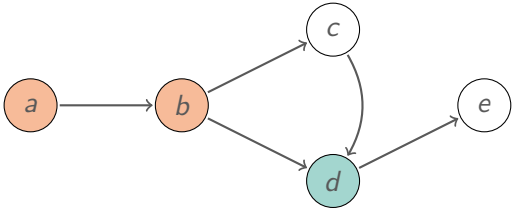
- Such cases can be naturally encoded as search problems in a state space, where the agent searches for an optimal path, p, from a starting state, s, to a goal state, g.
- Thus, the *what* aspect corresponds to the goal state, g, the agent has to reach.
- Similarly, the *why* aspect corresponds to the shape of the optimal path, p.
- Essentially, by applying restrictions on the shape of that path, the agent is searching for a tuple, (p, g), such that p connects its starting state, s, with the end state, g, while p adheres to a set of constraints, $\mathcal{C}$.

Argumentation Frameworks



$$\mathcal{A} = \{a, b, c, d, e\}$$
$$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$$

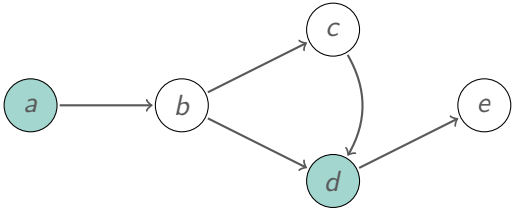
Argumentation Frameworks



$$\mathcal{A} = \{a, b, c, d, e\}$$
$$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$$

- An argument $x \in \mathcal{A}$ is **acceptable** wrt $E \subseteq \mathcal{A}$ if $\forall y \in \mathcal{A} \exists z \in E((y, x) \in \mathcal{R} \rightarrow (z, x) \in \mathcal{R})$.

Argumentation Frameworks

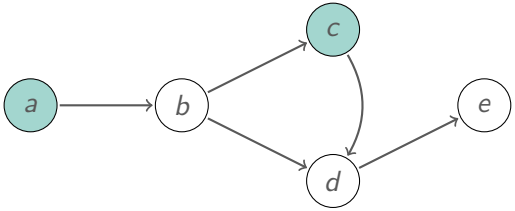


$\mathcal{A} = \{a, b, c, d, e\}$

$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$

- An argument $x \in \mathcal{A}$ is **acceptable** wrt $E \subseteq \mathcal{A}$ if $\forall y \in \mathcal{A} \exists z \in E((y, x) \in \mathcal{R} \rightarrow (z, x) \in \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **conflict free** if $\forall x, y \in E \ ((x, y) \notin \mathcal{R})$.

Argumentation Frameworks

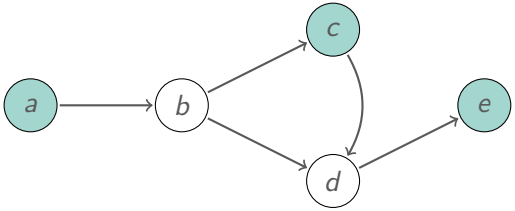


$\mathcal{A} = \{a, b, c, d, e\}$

$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$

- An argument $x \in \mathcal{A}$ is **acceptable** wrt $E \subseteq \mathcal{A}$ if $\forall y \in \mathcal{A} \exists z \in E((y, x) \in \mathcal{R} \rightarrow (z, x) \in \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **conflict free** if $\forall x, y \in E\ ((x, y) \notin \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **admissible** if it is **conflict free** and each $x \in E$ is **acceptable** wrt E.

Argumentation Frameworks

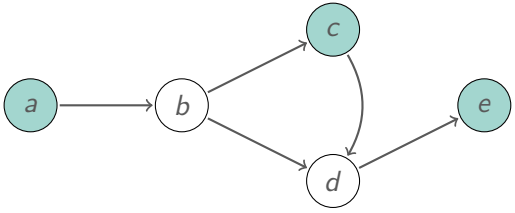


$\mathcal{A} = \{a, b, c, d, e\}$

$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$

- An argument $x \in \mathcal{A}$ is **acceptable** wrt $E \subseteq \mathcal{A}$ if $\forall y \in \mathcal{A} \exists z \in E((y, x) \in \mathcal{R} \rightarrow (z, x) \in \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **conflict free** if $\forall x, y \in E\ ((x, y) \notin \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **admissible** if it is **conflict free** and each $x \in E$ is **acceptable** wrt E.
- E is a **complete** extension of $\langle \mathcal{A}, \mathcal{R} \rangle$ if it is admissible & if x is acceptable wrt E then $x \in E$.

Argumentation Frameworks



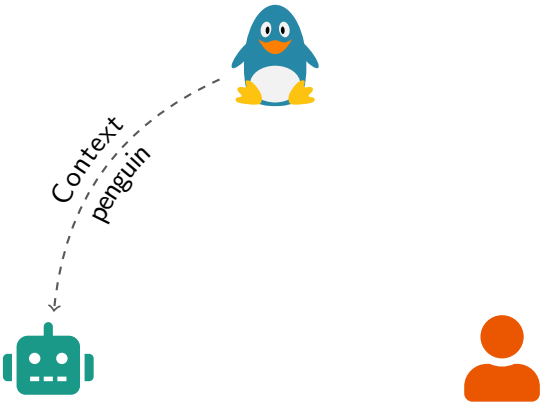
$\mathcal{A} = \{a, b, c, d, e\}$

$\mathcal{R} = \{(a, b), (b, c), (c, d), (b, d), (d, c), (d, e)\}$

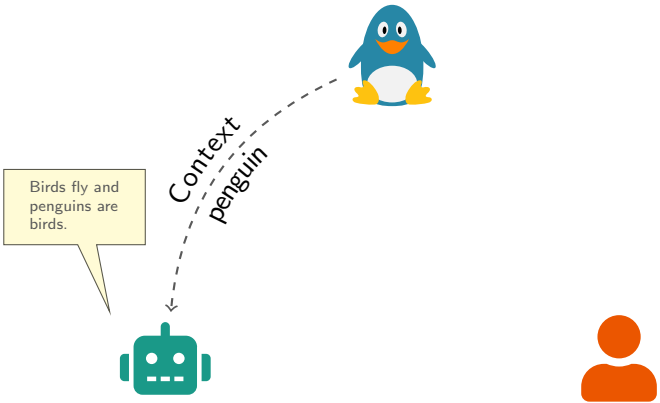
- An argument $x \in \mathcal{A}$ is **acceptable** wrt $E \subseteq \mathcal{A}$ if $\forall y \in \mathcal{A} \exists z \in E((y, x) \in \mathcal{R} \rightarrow (z, x) \in \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **conflict free** if $\forall x, y \in E\ ((x, y) \notin \mathcal{R})$.
- A set $E \subseteq \mathcal{A}$ is **admissible** if it is **conflict free** and each $x \in E$ is **acceptable** wrt E.
- E is a **complete** extension of $\langle \mathcal{A}, \mathcal{R} \rangle$ if it is admissible & if x is acceptable wrt E then $x \in E$.
- E is a **grounded** extension of $\langle \mathcal{A}, \mathcal{R} \rangle$ if it is the smallest complete extension of $\langle \mathcal{A}, \mathcal{R} \rangle$.

Machine Coaching

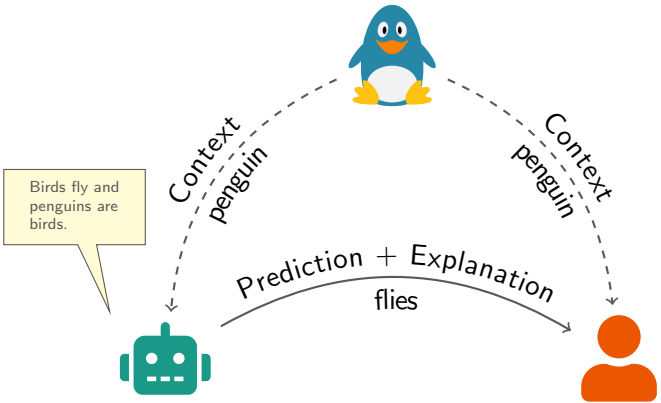
Machine Coaching



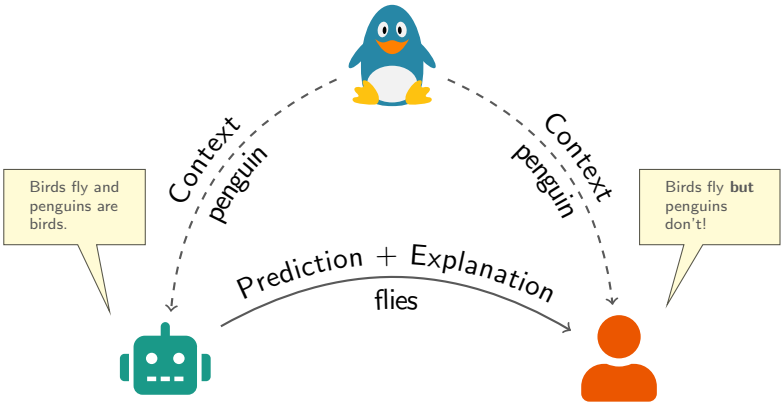
Machine Coaching



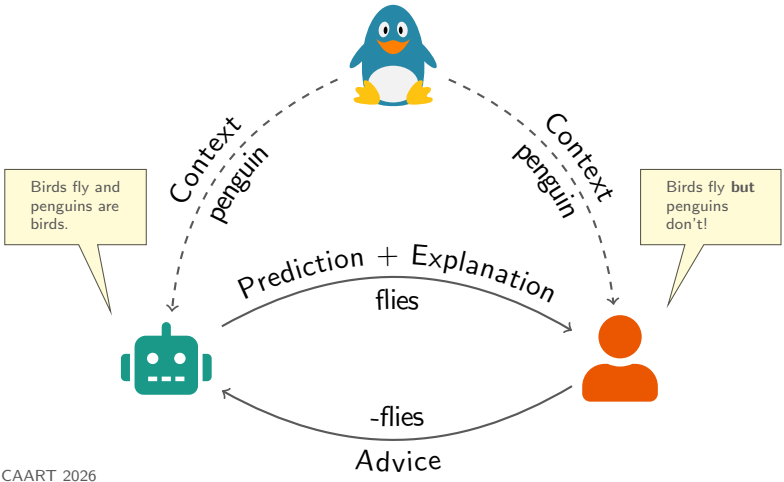
Machine Coaching



Machine Coaching



Machine Coaching



Machine Coaching

- Argumentation-based interaction between the coach and the machine.

Machine Coaching

- Argumentation-based interaction between the coach and the machine.
- Exception-based knowledge representation.

Machine Coaching

- Argumentation-based interaction between the coach and the machine.
- Exception-based knowledge representation.
- Transparent interaction.

Machine Coaching

- Argumentation-based interaction between the coach and the machine.
- Exception-based knowledge representation.
- Transparent interaction.
- Efficient reasoning.

Machine Coaching

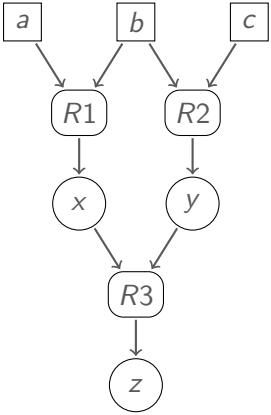
- Argumentation-based interaction between the coach and the machine.
- Exception-based knowledge representation.
- Transparent interaction.
- Efficient reasoning.
- Provably efficacious and efficient under PAC semantics.

Machine Coaching

- Argumentation-based interaction between the coach and the machine.
- Exception-based knowledge representation.
- Transparent interaction.
- Efficient reasoning.
- Provably efficacious and efficient under PAC semantics.
- Empirically verified learning efficiency and efficacy.

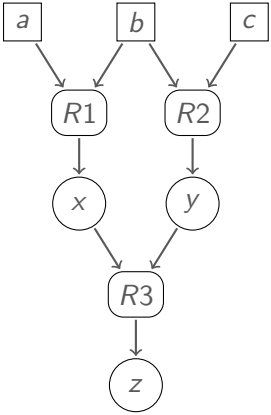
Machine Coaching and AFs

- **Rule-based Representation**: Knowledge is represented through rules.



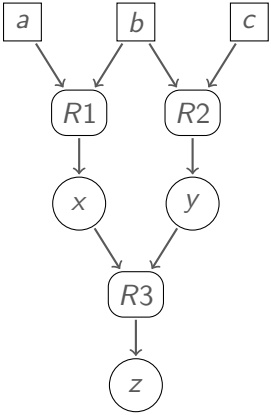
Machine Coaching and AFs

- **Rule-based Representation**: Knowledge is represented through rules.
- **Structured Arguments**: Trees of chained rules supporting an inference.



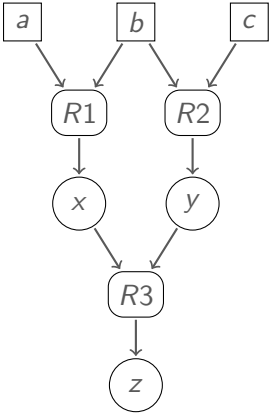
Machine Coaching and AFs

- **Rule-based Representation**: Knowledge is represented through rules.
- **Structured Arguments**: Trees of chained rules supporting an inference.
- **Grounded Semantics**: Cautious reasoning given a certain set of rules.



Machine Coaching and AFs

- **Rule-based Representation**: Knowledge is represented through rules.
- **Structured Arguments**: Trees of chained rules supporting an inference.
- **Grounded Semantics**: Cautious reasoning given a certain set of rules.
- **Efficient Reasoning**: Efficiency both in terms of arguments as well as in terms of available rules.

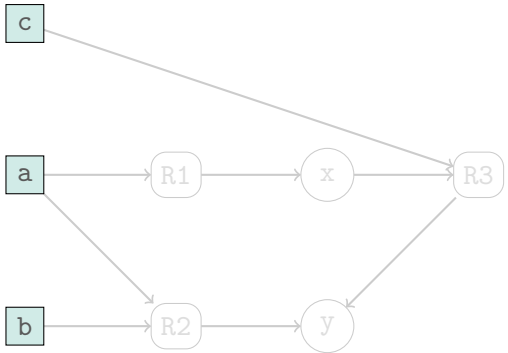


Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```

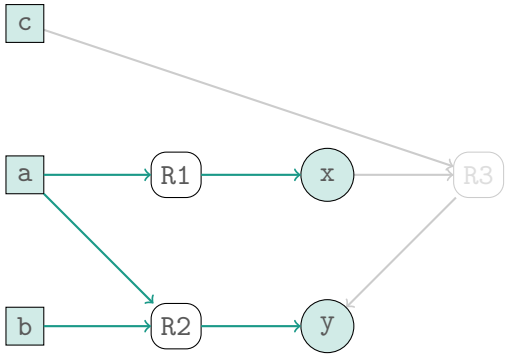
Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```



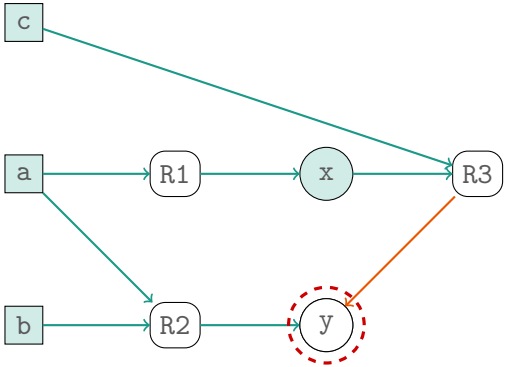
Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```



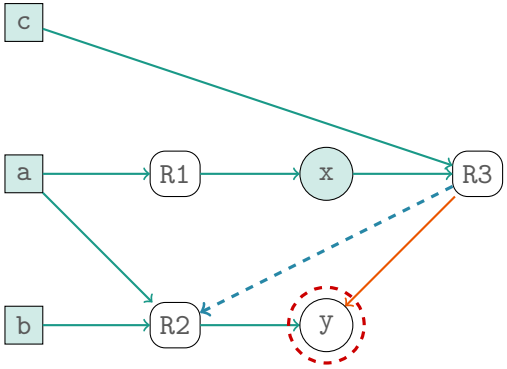
Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```



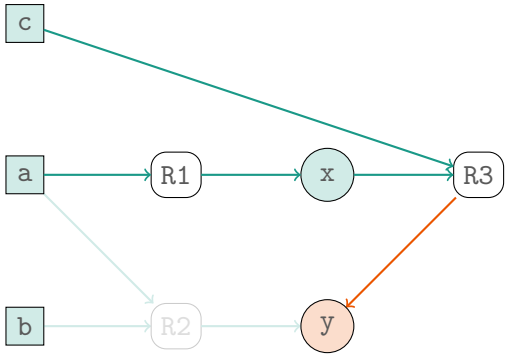
Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```



Reasoning in Machine Coaching

```
Context
a; b; c;

@Knowledge
R1 :: a implies x;
R2 :: a, b implies y;
R3 :: x, c implies -y;

@Priorities
R3 > R2
```



Search Coaching

- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.

Search Coaching

- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.
- We explore the highly structured problem of sorting a randomly permuted list of n integers by gradually providing advice, based on some common sorting algorithms.

Search Coaching

- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.
- We explore the highly structured problem of sorting a randomly permuted list of n integers by gradually providing advice, based on some common sorting algorithms.
- Our goal is to explore:

Search Coaching

- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.
- We explore the highly structured problem of sorting a randomly permuted list of n integers by gradually providing advice, based on some common sorting algorithms.
- Our goal is to explore:
  - whether the agent learns how to sort (the *what*);

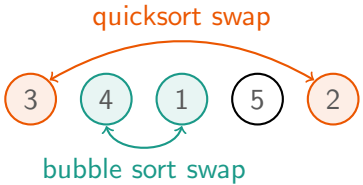
Search Coaching

- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.
- We explore the highly structured problem of sorting a randomly permuted list of n integers by gradually providing advice, based on some common sorting algorithms.
- Our goal is to explore:
  - whether the agent learns how to sort (the *what*);
  - whether the agent sorts faithfully to its coach's policy (the *how*), and;

Search Coaching

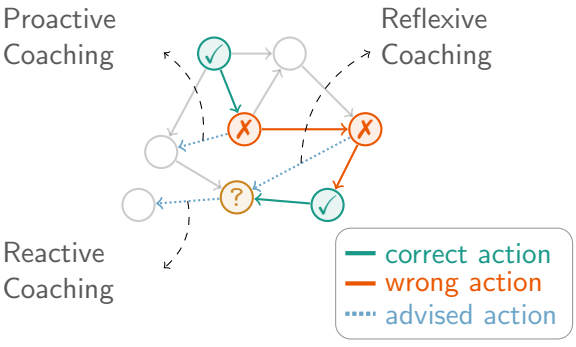
- In this work, we use Machine Coaching to gradually instill knowledge to an agent about both *what* and *how* to look for a goal state g.
- We explore the highly structured problem of sorting a randomly permuted list of n integers by gradually providing advice, based on some common sorting algorithms.
- Our goal is to explore:
  - whether the agent learns how to sort (the *what*);
  - whether the agent sorts faithfully to its coach's policy (the *how*), and;
  - how efficient the learning process is.

Sorting Rules




```
Rbf :: (3,4,1,5,2) implies swap(4,1);
Rbp :: (_,4,1,_,_) implies swap(4,1);
Rqf :: (3,4,1,5,2) implies swap(3,2);
Rqp :: (3,_,_,_,2) implies swap(3,2);
```

Coaching Styles



Coaching Process

Data: s, g, *learner*, *coach*
Result: *learner* with updated search policy; *path* from s to g.
do
    $path \leftarrow learner.search\_path(s, g)$;
    $advice \leftarrow coach.evaluate(path, s, g)$;
    $learner.update\_hypothesis(advice)$;
while *advice is not* `null`;

Definitions

- A *coaching step* is a single advice exchange between the coach and the agent.

Definitions

- A *coaching step* is a single advice exchange between the coach and the agent.
- *Advice coverage* is defined to be the percentage of information included in a rule's body compared to state size, n.

Definitions

- A *coaching step* is a single advice exchange between the coach and the agent.
- *Advice coverage* is defined to be the percentage of information included in a rule's body compared to state size, n.
- *Memory* in our context corresponds to the ability of the agent to recall previous from potentially structurally different settings.

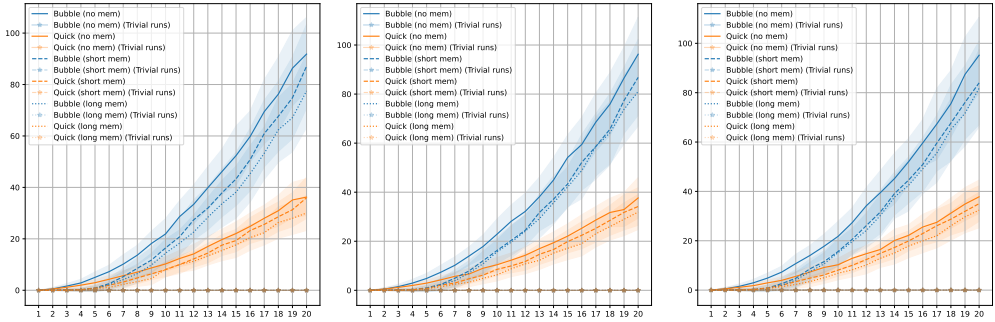
Definitions

- A *coaching step* is a single advice exchange between the coach and the agent.
- *Advice coverage* is defined to be the percentage of information included in a rule's body compared to state size, n.
- *Memory* in our context corresponds to the ability of the agent to recall previous from potentially structurally different settings.
- *Coaching conformity* is the extent to which the agent's optimal path p corresponds to what the coaching policy would yield as an optimal path within a coaching session.

Experiments Run

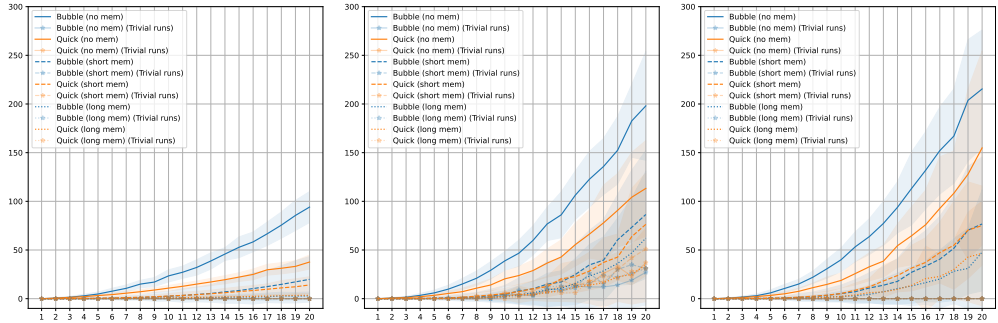
Attribute	Series A Experiments	Series B Experiments
State size, n	$\{1, 2, \ldots, 20\}$	$\{5, 10, 15, 20\}$
Iterations, m	100	20
Advice type	Full / Partial	partial@k
Memory	long / short / no	
Coaching style	reactive / reflexive / proactive	
Coaching policy	bubble sort, quicksort	

Series A: Full Advice Coaching Steps



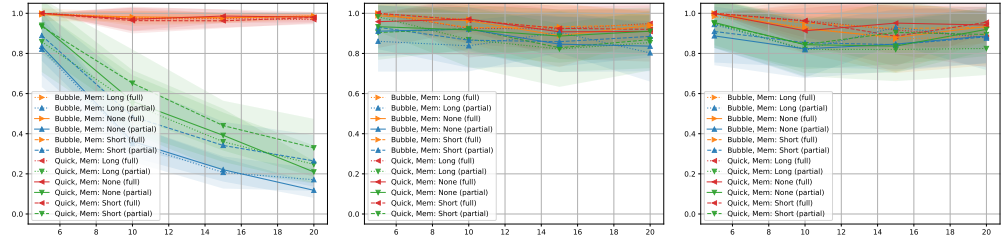
(a) Reactive (b) Reflexive (c) Proactive

Series A: Partial Advice Coaching Steps



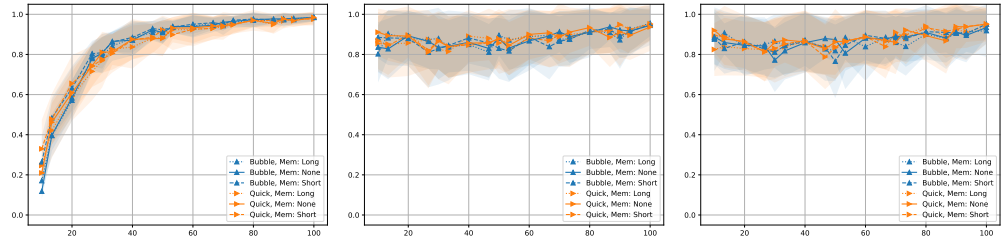
(a) Reactive (b) Reflexive (c) Proactive

Series B: Coaching Conformity vs State Size, n



(a) Reactive (b) Reflexive (c) Proactive

Series B: Coaching Conformity vs Advice Coverage



(a) Reactive (b) Reflexive (c) Proactive

Discussion & Future Work

- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.

Discussion & Future Work

- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.
- Moreover, we have demonstrated that under the coaching paradigm, one can fine tune the agent's behavioral conformity by properly adjusting the coaching protocol.

Discussion & Future Work

- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.
- Moreover, we have demonstrated that under the coaching paradigm, one can fine tune the agent's behavioral conformity by properly adjusting the coaching protocol.
- Next steps in this exploration might include but not restrict to:

Discussion & Future Work

- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.
- Moreover, we have demonstrated that under the coaching paradigm, one can fine tune the agent's behavioral conformity by properly adjusting the coaching protocol.
- Next steps in this exploration might include but not restrict to:
  - the exploration of other toy examples of higher complexity;

Discussion & Future Work

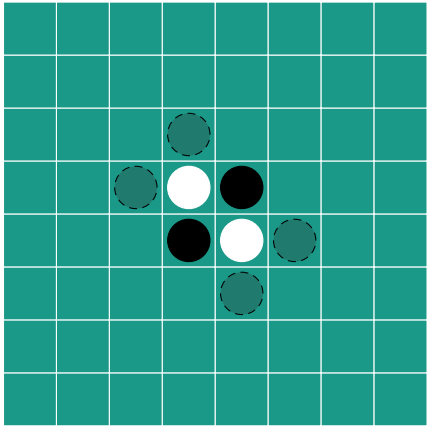
- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.
- Moreover, we have demonstrated that under the coaching paradigm, one can fine tune the agent's behavioral conformity by properly adjusting the coaching protocol.
- Next steps in this exploration might include but not restrict to:
  - the exploration of other toy examples of higher complexity;
  - comparison with other Machine Learning approaches, e.g., Reinforcement Learning;

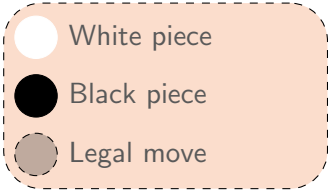
Discussion & Future Work

- In our exploration, effectively coaching an agent to perform a certain sorting task using purely symbolic means has proven to be feasible.
- Moreover, we have demonstrated that under the coaching paradigm, one can fine tune the agent's behavioral conformity by properly adjusting the coaching protocol.
- Next steps in this exploration might include but not restrict to:
  - the exploration of other toy examples of higher complexity;
  - comparison with other Machine Learning approaches, e.g., Reinforcement Learning;
  - application in realistic settings.

Appendix

How to play Othello?

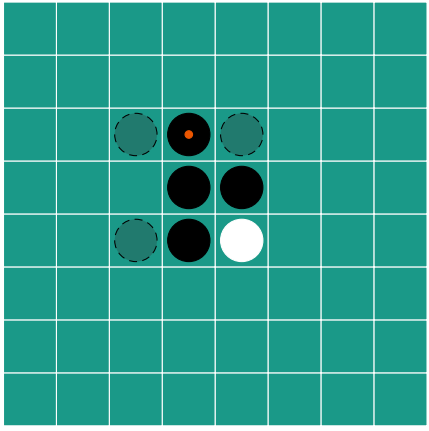




Rules

- Black plays first.
- Any new piece may be placed so as to enclose consecutive opponent pieces within friendly ones on the same row/column/diagonal.
- Any enclosed pieces are "converted" to the opponent's color.
- The player with the most pieces once the board is full or no more legal moves are allowed for any side wins.

How to play Othello?



White piece

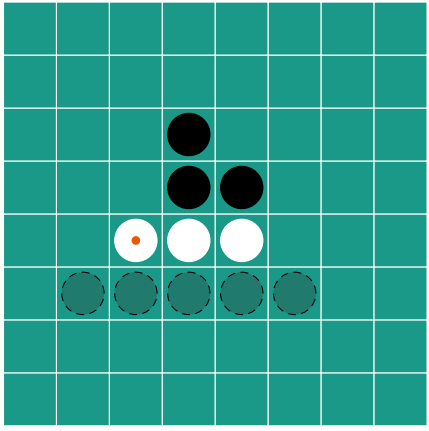
Black piece

Legal move

Rules

- Black plays first.
- Any new piece may be placed so as to enclose consecutive opponent pieces within friendly ones on the same row/column/diagonal.
- Any enclosed pieces are "converted" to the opponent's color.
- The player with the most pieces once the board is full or no more legal moves are allowed for any side wins.

How to play Othello?



White piece

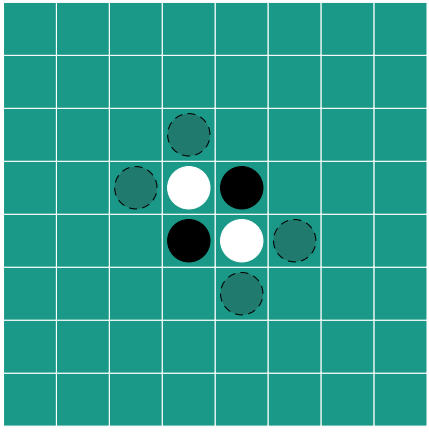
Black piece

Legal move

Rules

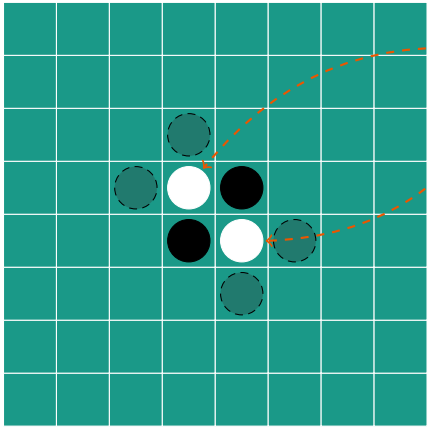
- Black plays first.
- Any new piece may be placed so as to enclose consecutive opponent pieces within friendly ones on the same row/column/diagonal.
- Any enclosed pieces are "converted" to the opponent's color.
- The player with the most pieces once the board is full or no more legal moves are allowed for any side wins.

Othello's Language



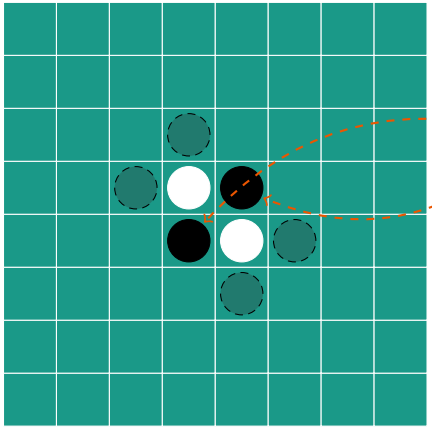
```
Context
cell(3, 3, 1);
cell(4, 4, 1);
cell(3, 4, -1);
cell(4, 3, -1);
legalMove(2, 3);
legalMove(3, 2);
legalMove(4, 5);
legalMove(5, 4);
cell(0, 0, 0); ...; cell(7, 7, 0);
```

Othello's Language



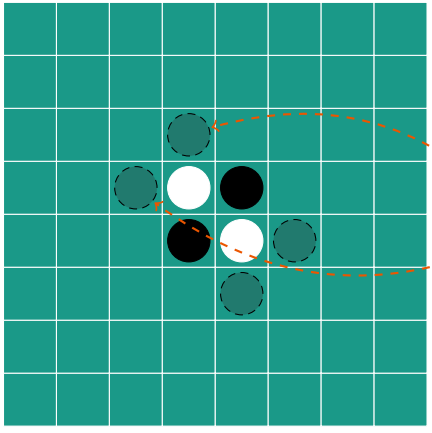
```
Context
cell(3, 3, 1);
cell(4, 4, 1);
cell(3, 4, -1);
cell(4, 3, -1);
legalMove(2, 3);
legalMove(3, 2);
legalMove(4, 5);
legalMove(5, 4);
cell(0, 0, 0); ...; cell(7, 7, 0);
```

Othello's Language



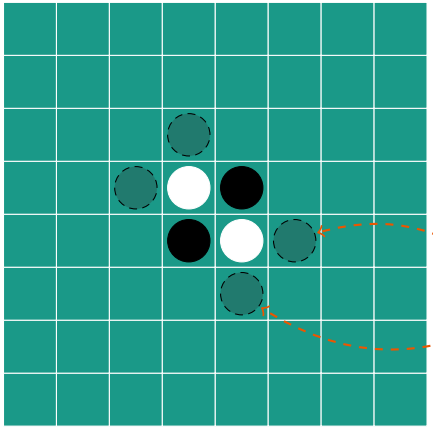
```
Context
cell(3, 3, 1);
cell(4, 4, 1);
cell(3, 4, -1);
cell(4, 3, -1);
legalMove(2, 3);
legalMove(3, 2);
legalMove(4, 5);
legalMove(5, 4);
cell(0, 0, 0); ...; cell(7, 7, 0);
```

Othello's Language



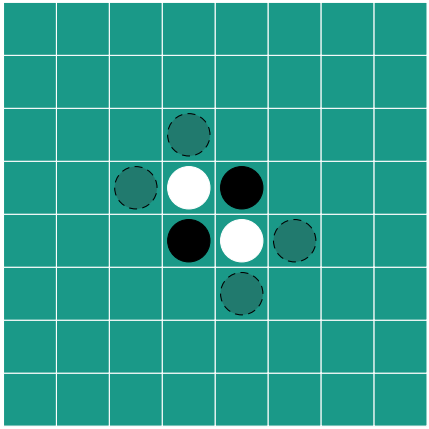
```
Context
cell(3, 3, 1);
cell(4, 4, 1);
cell(3, 4, -1);
cell(4, 3, -1);
legalMove(2, 3);
legalMove(3, 2);
legalMove(4, 5);
legalMove(5, 4);
cell(0, 0, 0); ...; cell(7, 7, 0);
```

Othello's Language



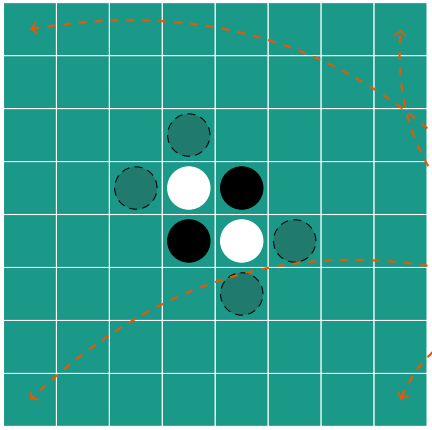
```
Context
cell(3, 3, 1);
cell(4, 4, 1);
cell(3, 4, -1);
cell(4, 3, -1);
legalMove(2, 3);
legalMove(3, 2);
legalMove(4, 5);
legalMove(5, 4);
cell(0, 0, 0); ...; cell(7, 7, 0);
```

A Dummy Policy



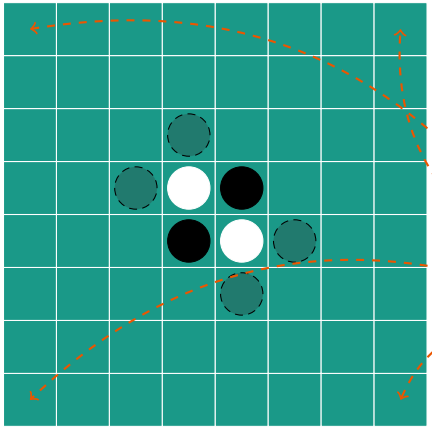
```
@Knowledge
D1 :: legalMove(X,Y) implies move(X,Y);
```

Simple Heuristics



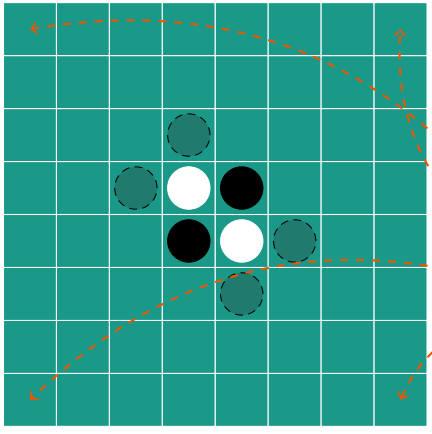
```
@Knowledge
D1 :: legalMove(X,Y) implies move(X,Y);
D2 :: legalMove(X,Y) implies -move(X,Y);

R1 :: legalMove(0,0) implies move(0,0);
R2 :: legalMove(0,7) implies move(0,7);
R3 :: legalMove(7,0) implies move(7,0);
R4 :: legalMove(7,7) implies move(7,7);
```

Simple Heuristics



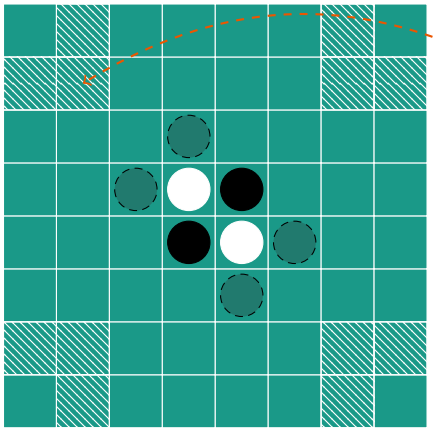
```
@Knowledge
D1 :: legalMove(X,Y) implies move(X,Y);
D2 :: legalMove(X,Y) implies -move(X,Y);

C1 :: implies corner(0,0);
C2 :: implies corner(0,7);
C3 :: implies corner(7,0);
C4 :: implies corner(7,7);
```

Simple Heuristics



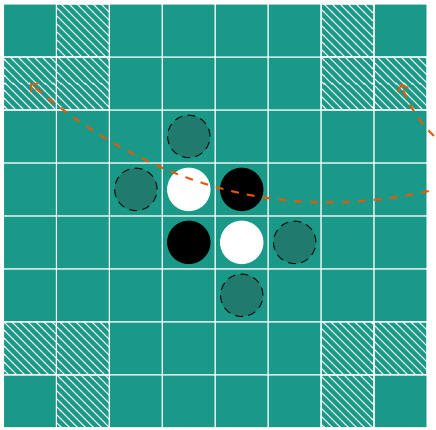
```
@Knowledge
D1 :: legalMove(X,Y) implies move(X,Y);
D2 :: legalMove(X,Y), legalMove(Z,W),
      corner(Z,W), -?=(X,Z), -?=(Y,W)
      implies -move(X,Y);
C1 :: implies corner(0,0);
C2 :: implies corner(0,7);
C3 :: implies corner(7,0);
C4 :: implies corner(7,7);
R1 :: legalMove(X,Y), corner(X,Y) implies
      move(X,Y);
```

More Complex Heuristics



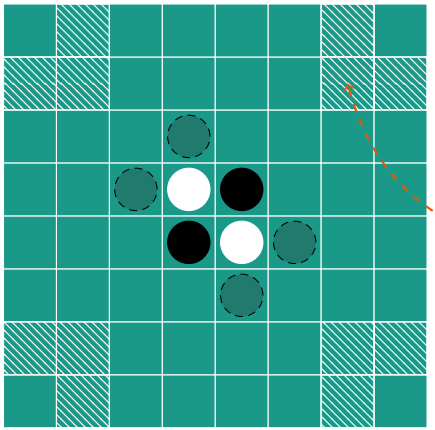
```
@Knowledge
...
A1 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y+1)
      implies adjC(Z,W);
```

More Complex Heuristics



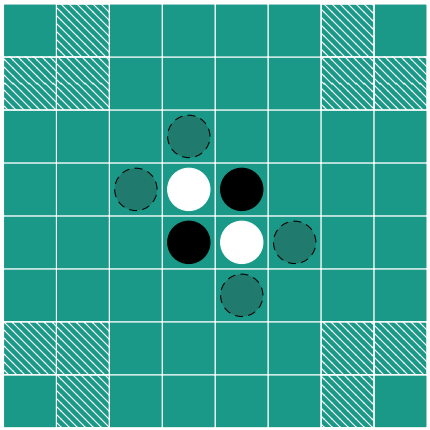
```
@Knowledge
...
A1 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y+1)
      implies adjC(Z,W);
A2 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y)
      implies adjC(Z,W);
```

More Complex Heuristics



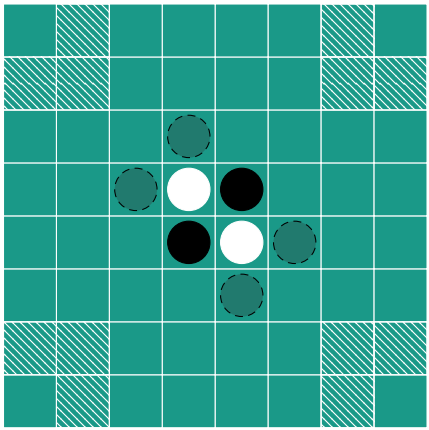
```
@Knowledge
...
A1 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y+1)
      implies adjC(Z,W);
A2 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y)
      implies adjC(Z,W);
A3 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y-1)
      implies adjC(Z,W);
```

More Complex Heuristics



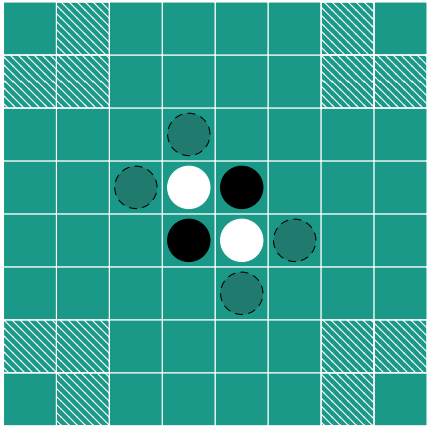
```
@Knowledge
...
A1 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y+1)
      implies adjC(Z,W);
A2 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y)
      implies adjC(Z,W);
A3 :: corner(X,Y), ?=(Z,X+1), ?=(W,Y-1)
      implies adjC(Z,W);
...
```

More Complex Heuristics



```
@Knowledge
...
@Procedures
function isAdj(x,y,z,w) {
    const X = parseInt(x);
    const Y = parseInt(y);
    const Z = parseInt(z);
    const W = parseInt(w);
    return Z-X < 2 && X-Z < 2 && W-Y < 2 &&
    Y-W < 2 && (X != Z || Y != W);
}
```

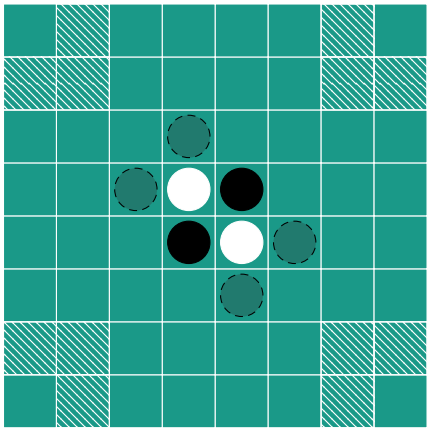
More Complex Heuristics



```
@Knowledge
...
@Procedures
function isAdj(x,y,z,w) {
    const X = parseInt(x);
    const Y = parseInt(y);
    const Z = parseInt(z);
    const W = parseInt(w);
    return Z-X < 2 && X-Z < 2 && W-Y < 2 &&
    Y-W < 2 && (X != Z || Y != W);
}
```

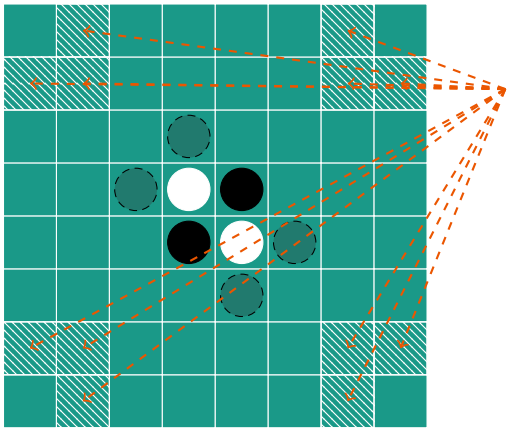
JavaScript

More Complex Heuristics



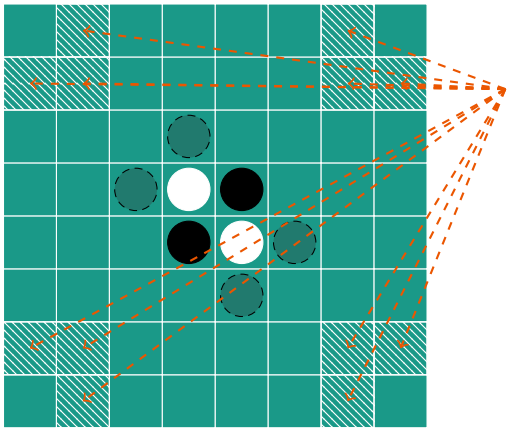
```
@Knowledge
...
R2 :: corner(X,Y), legalMove(Z,W),
      ?isAdj(X,Y,Z,W) implies -move(Z,W);
...
@Procedures
...
```

More Complex Heuristics



```
@Knowledge
...
R2 :: corner(X,Y), legalMove(Z,W),
      ?isAdj(X,Y,Z,W) implies -move(Z,W);
...
@Procedures
...
```

More Complex Heuristics



```
@Knowledge
...
R2 :: corner(X,Y), legalMove(Z,W),
      ?isAdj(X,Y,Z,W) implies -move(Z,W);
...
@Procedures
...
```

Benefits:

- More compact representation.
- Faster compilation.

Thank You!

Any Questions?